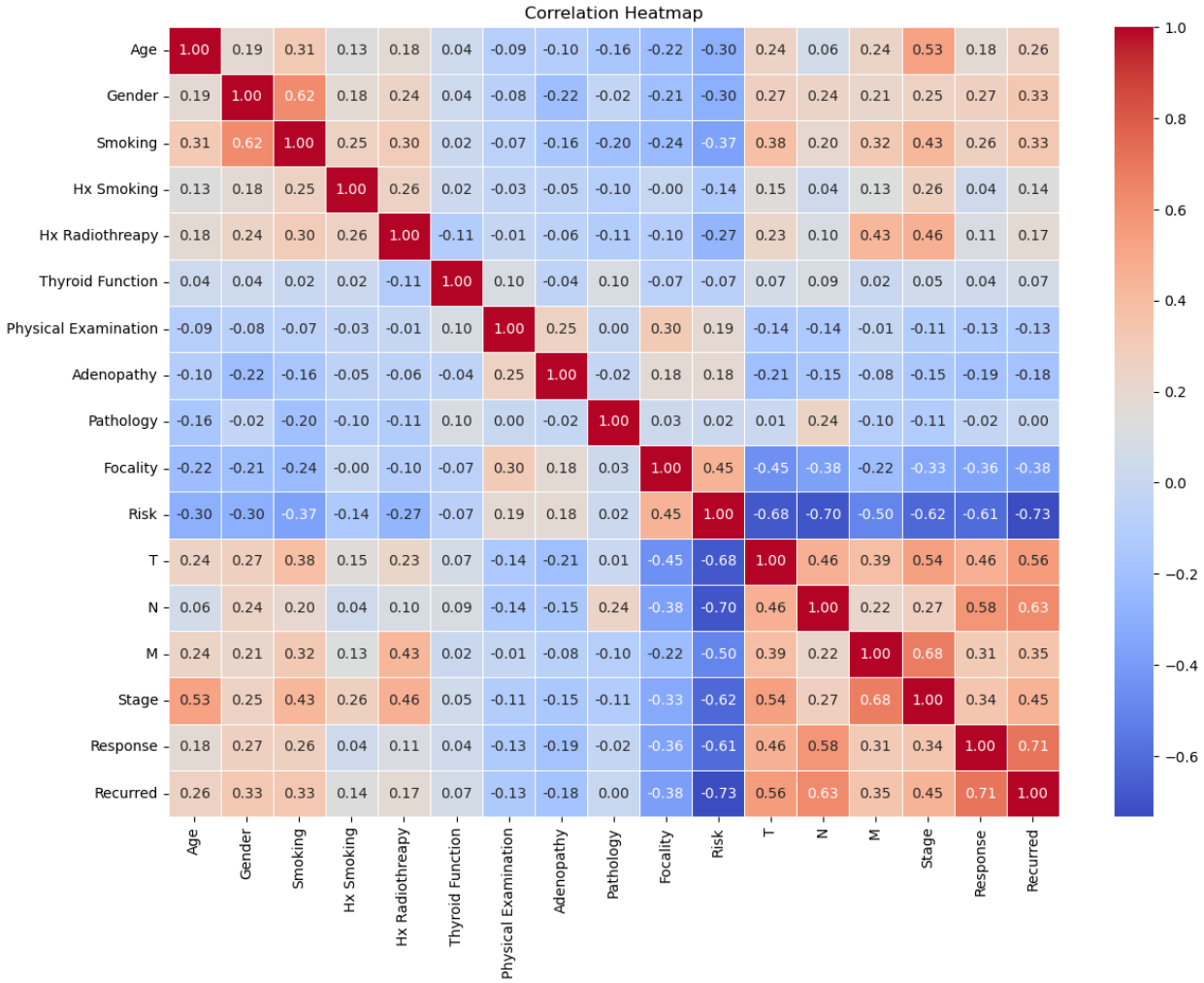
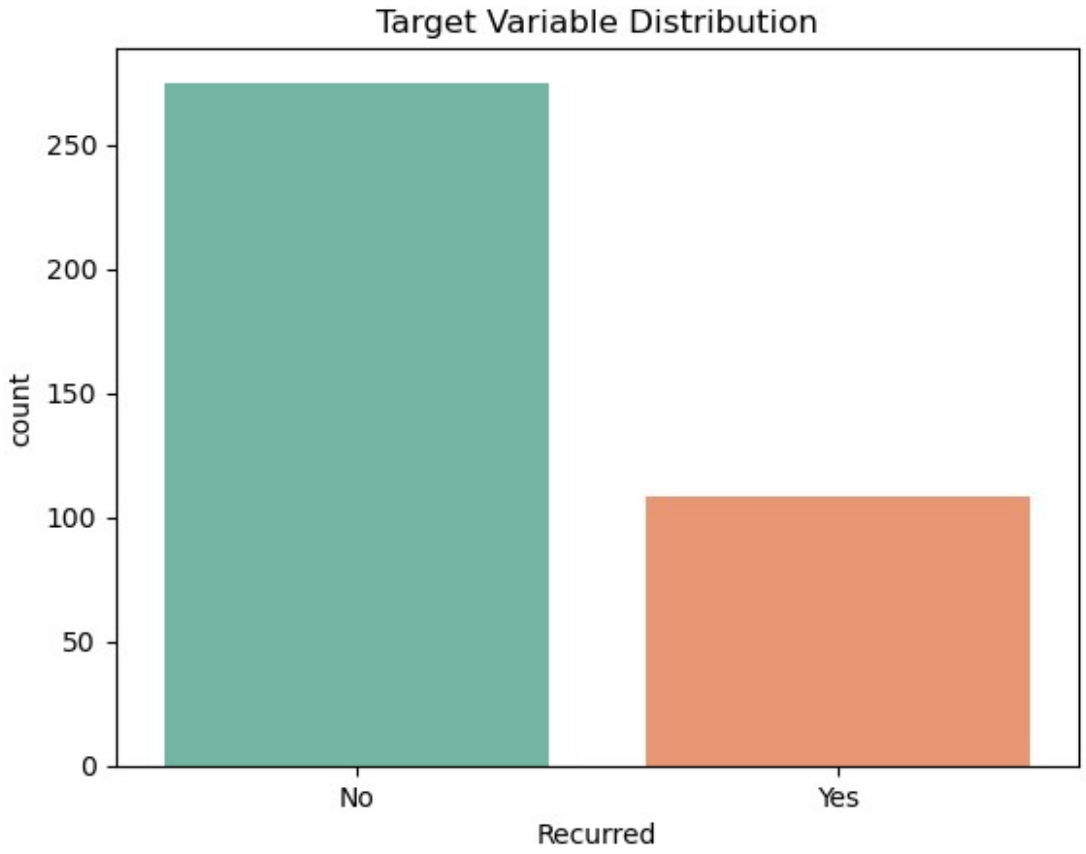
```
# Importing Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

# Load Dataset
df =
pd.read_csv(r'/Users/shreyash/Downloads/Differentiated_Thyroid_ancer_R
ecurrence.csv')


# EDA: Check for missing values
print("Missing values:\n", df.isnull().sum())


# Heatmap - encoding - temporarily
df_encoded = df.copy()
le_temp = LabelEncoder()
for col in df_encoded.columns:
    if df_encoded[col].dtype == 'object':
        df_encoded[col] = le_temp.fit_transform(df_encoded[col])

plt.figure(figsize=(14, 10))
sns.heatmap(df_encoded.corr(), annot=True, cmap='coolwarm', fmt='.2f',
linewidths=0.5)
plt.title('Correlation Heatmap')
plt.show()


# Target distribution
sns.countplot(x='Recurred', data=df, palette='Set2')
plt.title("Target Variable Distribution")
plt.show()
```

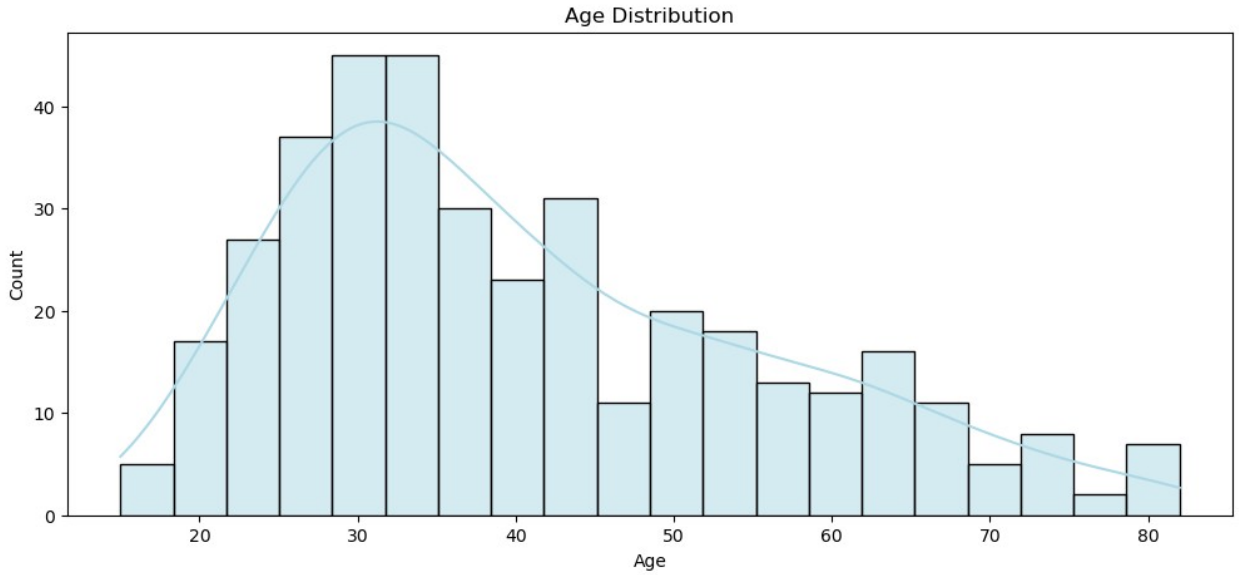
```
# outlier detection for 'Age'
plt.figure(figsize=(12, 5))
sns.histplot(df['Age'], kde=True, bins=20, color='lightblue')
plt.title('Age Distribution')
plt.show()

sns.boxplot(x='Recurred', y='Age', data=df, palette='Set2')
plt.title('Boxplot of Age by Recurred')
plt.show()



# Encode variables
le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])



# Features and Target
X = df.drop('Recurred', axis=1)
y = df['Recurred']

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Function to train and evaluate models
def train_evaluate_model(model, name):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print(f"\n{name} Results:")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("Classification Report:\n", classification_report(y_test,
y_pred))

# Logistic Regression
train_evaluate_model(LogisticRegression(max_iter=1000), "Logistic
Regression")

# Decision Tree
train_evaluate_model(DecisionTreeClassifier(), "Decision Tree")

# Random Forest
train_evaluate_model(RandomForestClassifier(), "Random Forest")

# K-Nearest Neighbors
train_evaluate_model(KNeighborsClassifier(), "K-Nearest Neighbors")
```

```
Missing values:
 Age                     0
Gender                   0
Smoking                  0
Hx Smoking               0
Hx Radiothreapy          0
Thyroid Function         0
Physical Examination     0
Adenopathy               0
Pathology                0
Focality                 0
Risk                     0
T                        0
N                        0
M                        0
Stage                    0
Response                 0
Recurred                 0
dtype: int64
```

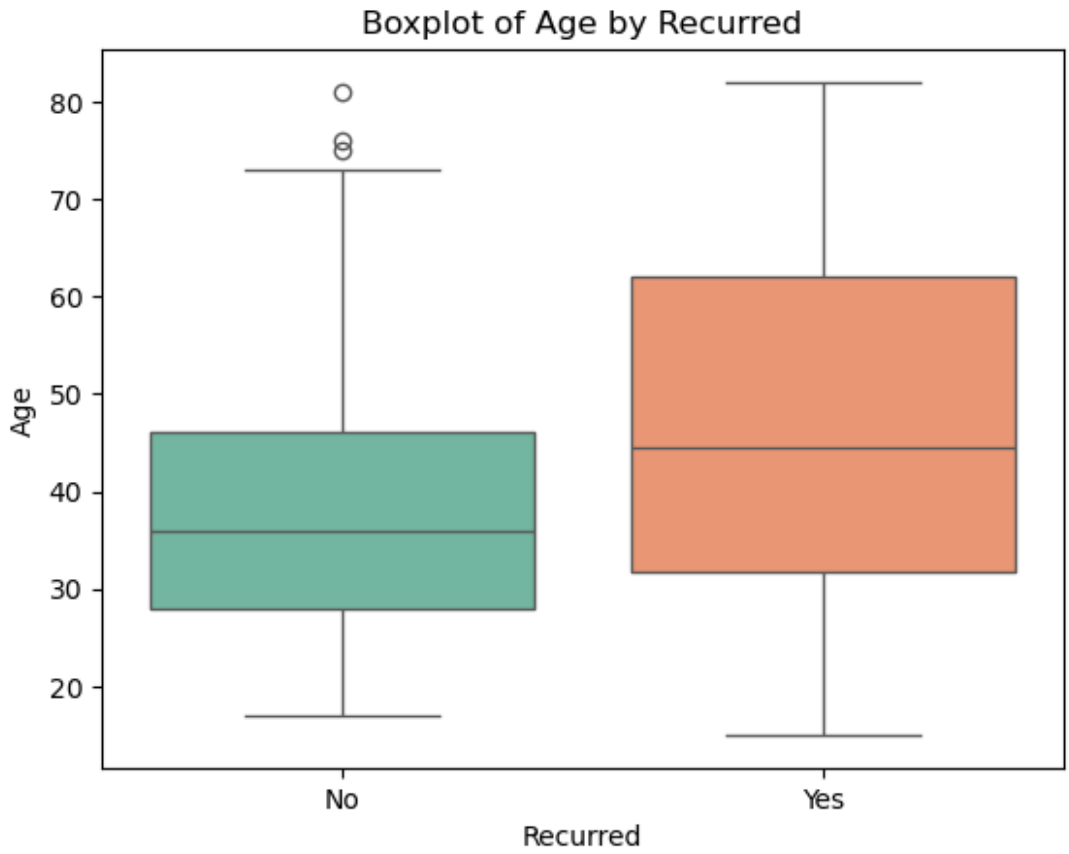


```
/var/folders/pp/r0x38d6x21s5r8kwt5qkdmy00000gn/T/
ipykernel_1590/4293066308.py:32: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.countplot(x='Recurred', data=df, palette='Set2')
```






```
/var/folders/pp/r0x38d6x21s5r8kwt5qkdmy00000gn/T/
ipykernel_1590/4293066308.py:42: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.boxplot(x='Recurred', y='Age', data=df, palette='Set2')
```




```
Logistic Regression Results:
Accuracy: 0.935064935064935
Confusion Matrix:
 [[57  1]
 [ 4 15]]
Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.98      0.96        58
           1       0.94      0.79      0.86        19

    accuracy                           0.94        77
   macro avg       0.94      0.89      0.91        77
weighted avg       0.94      0.94      0.93        77


Decision Tree Results:
Accuracy: 0.935064935064935
Confusion Matrix:
 [[54  4]
 [ 1 18]]
Classification Report:
```

```
              precision    recall  f1-score   support

           0       0.98      0.93      0.96        58
           1       0.82      0.95      0.88        19

    accuracy                           0.94        77
   macro avg       0.90      0.94      0.92        77
weighted avg       0.94      0.94      0.94        77


Random Forest Results:
Accuracy: 0.987012987012987
Confusion Matrix:
 [[58  0]
 [ 1 18]]
Classification Report:
              precision    recall  f1-score   support

           0       0.98      1.00      0.99        58
           1       1.00      0.95      0.97        19

    accuracy                           0.99        77
   macro avg       0.99      0.97      0.98        77
weighted avg       0.99      0.99      0.99        77


K-Nearest Neighbors Results:
Accuracy: 0.8571428571428571
Confusion Matrix:
 [[57  1]
 [10  9]]
Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.98      0.91        58
           1       0.90      0.47      0.62        19

    accuracy                           0.86        77
   macro avg       0.88      0.73      0.77        77
weighted avg       0.86      0.86      0.84        77


""" Best Model: Random Forest Classifier
     •    Highest Accuracy (98.7%)
     •    Best Precision, Recall, and F1-score for both classes
     •    Especially strong at predicting the minority class
(Recurred = 1)
"""
```